

KOHLER AI Bathroom Designer & Planner

Prompts Documentation

"From bathroom constraints to a personalized, space-aware bathroom design."

This document describes a student prototype built for the KOHLER–MIT-WPU AI Research Lab Program. It does not claim access to any official KOHLER internal system, database, or documentation. All product data referenced in examples is synthetic/demo data created for this prototype -- see data/data_sources.md.

1. AI Architecture

Core principle: **Generative AI proposes. Deterministic constraint optimization validates.** The application uses exactly two LLM calls, each scoped to one job, and a deterministic Python engine (constraint checks, optimizer, sustainability calculator, floor-plan renderer) that is the sole source of truth for prices, dimensions, feasibility, and scores.

Stage	Driven by
Requirement extraction	AI (LLM), with a rule-based fallback
Product retrieval	Deterministic (catalog filters)
Constraint validation	Deterministic (budget / category / space / clearance)
Bundle optimization	Deterministic (exhaustive search + weighted scoring)
Sustainability analysis	Deterministic (arithmetic on user-set assumptions)
2D layout generation	Deterministic (same zone geometry as validation)
Explanation generation	AI (LLM), grounded by a numeric-verification guard, with a template fallback

The LLM cannot override the deterministic engine: its requirement-extraction output is re-validated by the same Pydantic schema the manual form uses, and its explanation output only ever narrates a bundle that has already been scored and validated -- there is no code path where LLM output feeds back into price, dimension, or feasibility logic. Full detail: docs/architecture.md.

2. System Instructions

There are exactly two system prompts in this application, each narrowly scoped to one job, defined in `src/ai/prompts.py`. There is no general-purpose “master” system prompt -- see `prompts/system_prompt.md` for the rationale (smaller blast radius, mismatched grounding needs, different output types).

- **REQUIREMENT_EXTRACTION_SYSTEM_PROMPT** — used by `src/ai/requirement_parser.py`
- **EXPLANATION_SYSTEM_PROMPT** — used by `src/ai/explanation.py`

3. Requirement Extraction Prompt

Purpose

Converts the customer's free-text bathroom description into structured fields, without designing, selecting products, or judging feasibility.

Full text

You are a requirement-extraction assistant for a bathroom-design prototype.
Your **ONLY** job is to convert the user's natural-language description of their

desired bathroom into the structured fields you were given a schema for. You are NOT designing the bathroom, NOT selecting products, and NOT deciding whether anything is feasible -- a separate deterministic system does that.

Rules:

- Extract only what the user actually said or clearly implied. Do not invent a budget, dimensions, or style the user did not mention.
- If the user gives dimensions in a different unit (e.g. meters), convert to feet. If they give budget in lakhs (1 lakh = 100,000) or crores, convert to plain INR.
- `required_categories` must only contain: `smart_toilet`, `faucet`, `thermostatic_shower`, `vanity`. Map synonyms sensibly, but only include a category the user actually asked for or is clearly required by context.
- `style` must be exactly one of: `minimalist_modern`, `classic_luxury`, `japanese_zen`, `contemporary`. Pick the closest match; default to `contemporary` if truly unclear.
- Never invent product names, prices, or specifications -- you have no access to the product catalog and must not reference specific products.

Inputs / Outputs

Input: the system prompt (fixed) + the customer's raw free-text description. **Output:** a Pydantic `_ExtractedRequirement` object, produced via OpenAI's structured-output feature (`client.chat.completions.parse(response_format=_ExtractedRequirement)`), then re-validated into the application's real `RequirementSpec`.

4. Explanation-Generation Prompt

Purpose

Narrates an already-selected, already-scored, already-validated product bundle in plain, customer-facing language -- it never decides anything, only explains.

Full text

You are a design-explanation assistant for a bathroom-design prototype.

You will be given a JSON object describing a bundle of products that has ALREADY been selected and validated by a separate deterministic system: the prices, dimensions, feasibility result, and scores are all final and correct. Your ONLY job is to explain, in plain and encouraging language, why this bundle suits the customer's stated requirements.

Hard rules -- violating any of these makes your output unusable:

1. Do not state any price, dimension, water-consumption value, or score that is not already present in the JSON you were given. Round for readability only, never invent or adjust a figure.
2. Do not claim the layout is construction-ready or that measurements are exact -- always defer final verification to a qualified professional.
3. Do not claim any certification, official KOHLER endorsement, or sustainability certification not stated in the input data.
4. If the bundle has validation violations or warnings, mention them.
5. Keep it concise: 3-5 short sentences, warm and professional in tone.

Inputs / Outputs

Input: the system prompt (fixed) + the bundle's scored JSON payload (`ScoredBundle.as_dict()`).

Output: 3–5 sentences of prose via a `plain_chat.completions.create()` call (no structured-output schema, since the target is text, not data).

5. Structured Output Schema

Used only by the requirement-extraction call. Deliberately a separate model from the domain RequirementSpec (plain Literals instead of Enums) for maximum compatibility with OpenAI's structured-output feature:

```
class _ExtractedBathroom(BaseModel):
    length_ft: float
    width_ft: float
    ceiling_height_ft: Optional[float] = None

class _ExtractedRequirement(BaseModel):
    bathroom: _ExtractedBathroom
    budget_inr: float
    style: Literal["minimalist_modern", "classic_luxury",
                  "japanese_zen", "contemporary"]
    required_categories: List[Literal["smart_toilet", "faucet",
                                      "thermostatic_shower", "vanity"]]
    priorities: List[str] = []
    water_saving_preference: bool = False
    accessibility_preference: bool = False
```

6. Grounding Rules

- **Extraction:** two independent layers -- (1) the OpenAI structured-output feature enforces the schema above at the API level; (2) the parsed result is re-validated by constructing the real RequirementSpec, which applies its own validators (e.g. `required_categories` must be non-empty). A schema-valid but semantically empty result is caught at layer 2.
- **Explanation:** grounded post-hoc by the numeric-verification guard (section 7) rather than by a schema, since the output is free text.
- **Both:** a model refusal is always treated as a failure, never as a valid-but-empty result.

7. Numerical Grounding / Validation Rules

Implemented in `src/ai/explanation.py`:

- `_collect_allowed_numbers(payload)` walks the entire input JSON and, for every number, adds it plus derived forms to an “allowed” set: rounded to 2dp, rounded to an integer, in thousands, in lakhs (raw and 2dp), and as a percentage (for 0–1 scores).
- `_verify_numbers_grounded(text, allowed, tolerance=0.03)` extracts every numeric substring from the model's output and checks it against that set within a small relative tolerance.

- If **any** number in the response can't be matched, the entire response is discarded (not edited) and the deterministic template is used instead.
- Verified directly by tests, including a fabricated-number case that is confirmed to actually get rejected -- not just designed to be.

8. Offline Fallback

Component	Fallback	Trigger
Requirement extraction	Regex/keyword rule-based parser	Any exception: no key, invalid key, rate limit, timeout, network error, malformed/invalid output
Explanation	Deterministic string-template summary	Any exception, OR a fabricated/unverifiable number caught by the grounding guard

Both fallbacks run with zero network access and zero API key, and the entire Streamlit app is smoke-tested end to end with no OPENAI_API_KEY set -- the “must work without a key” requirement is a verified, tested property.

9. Example Input / Output

Requirement extraction

Input (the brief's own demo sentence):

I have an 8 by 6 feet bathroom. My budget is Rs.2.5 lakh. I want a Japanese Zen style with a smart toilet, shower, vanity, and a minimalist faucet.

Output (structured -- verified identical whether produced by the LLM path or the rule-based fallback, for this exact sentence):

```
{
  "bathroom": {"length_ft": 8, "width_ft": 6, "ceiling_height_ft": null},
  "budget_inr": 250000,
  "style": "japanese_zen",
  "required_categories": ["smart_toilet", "faucet",
                        "thermostatic_shower", "vanity"],
  "priorities": [],
  "water_saving_preference": false,
  "accessibility_preference": false
}
```

Explanation (template fallback, reproduced from an actual test fixture)

Balanced Bundle combines smart_toilet (Prototype Smart Comfort Toilet with Bidet), faucet (Prototype Zen Minimal Faucet), thermostatic_shower (Prototype Premium Rain Shower Suite), vanity (Prototype Zen Floating Vanity) for a total of Rs.237,500, leaving Rs.12,500 of your stated budget unused. About 75% of the selected products are tagged for your requested style, and the products score 33% on the prototype's water-efficiency scale. This combination passed the prototype's budget, category, and simplified space checks, but is not a construction-ready plan.

10. Prompt Safety and Hallucination Controls

- Explicit “you are NOT deciding feasibility / selecting products” framing in the extraction prompt.
- Explicit “never invent a number not in the JSON you were given” rule in the explanation prompt, enforced in code by the numeric-grounding guard, not just by instruction.
- Two-layer schema validation on extraction output (structured-output schema, then RequirementSpec's own validators).
- A model refusal is treated as a failure and triggers the same fallback as any other error.
- Every fallback path is unit-tested without network access; the app's own end-to-end smoke test runs with no API key set.
- The application never claims access to an official KOHLER internal database, system, or documentation -- all example data in this document is clearly labeled synthetic/demo data from this prototype's own catalog.

Source of truth for everything in this document: `src/ai/prompts.py`, `src/ai/requirement_parser.py`, `src/ai/explanation.py`, and `prompts/*.md` in the project repository.